Aula 22 - Protocolos IoT MQTT/OPC e Web

Descrição: A aula apresenta os fundamentos e a arquitetura do protocolo IoT MQTT, detalhando seu modelo de publicação/assinatura, níveis de serviço (QoS), mecanismos de segurança e um roteiro prático de simulação.

---

1. Introdução aos Protocolos IoT e Comunicação Web

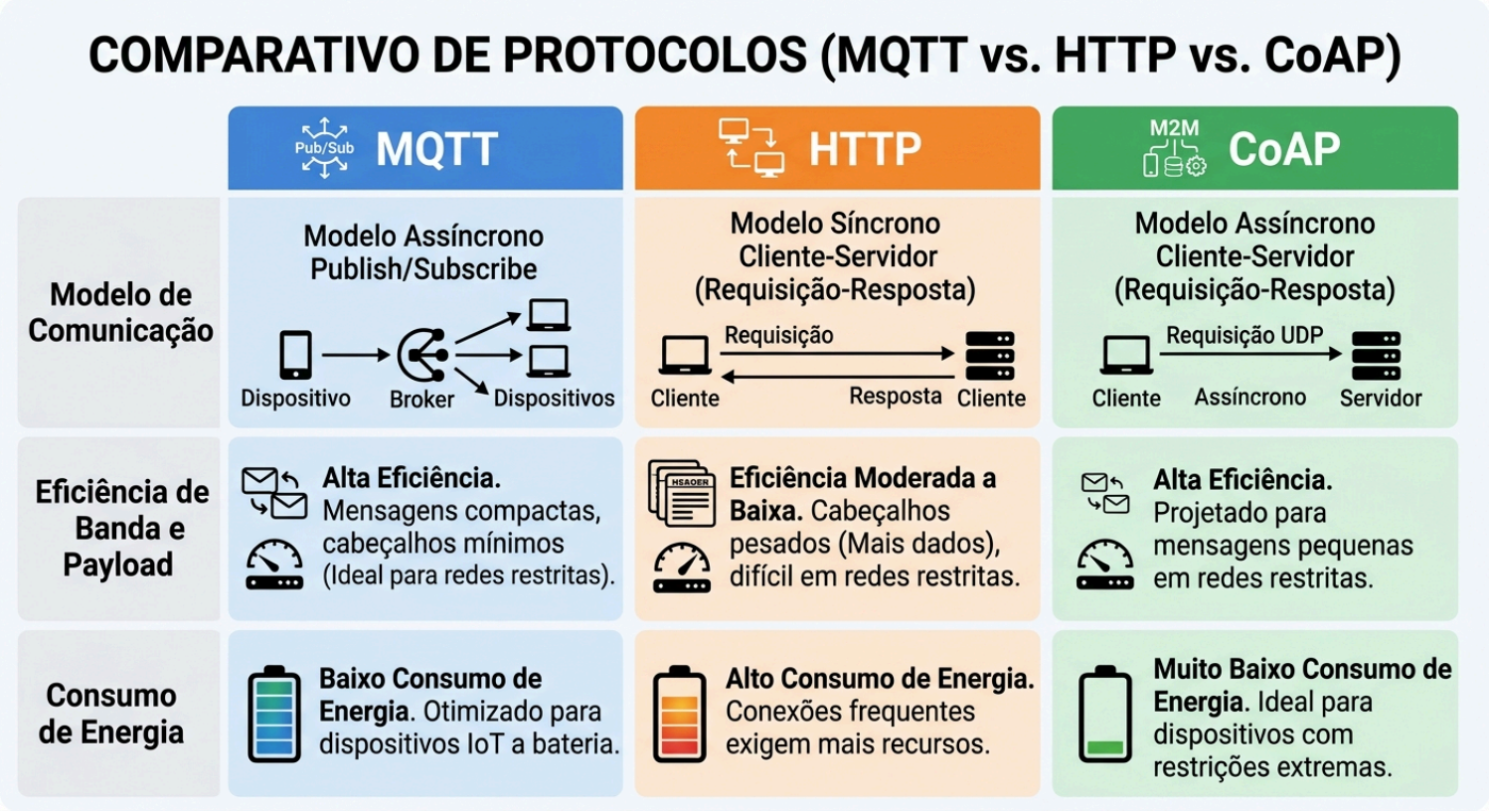
Conceito de Protocolos Leves para IoT

O MQTT (Message Queuing Telemetry Transport) é um protocolo de comunicação de padrão aberto, leve e eficiente. Ele foi especificamente projetado para operar em ambientes com recursos limitados, destacando-se em redes que apresentam baixa largura de banda, alta latência ou conectividade intermitente e instável. Desenvolvido inicialmente pela IBM nos anos 1990, o protocolo baseia-se em um modelo cliente/servidor sobre o TCP/IP usual e utiliza o paradigma assíncrono Publish/Subscribe (Publicar/Assinar), sendo orientado a eventos. Nesse modelo, os dispositivos atuam como clientes e se conectam a um servidor central, o broker, que é responsável por gerenciar a comunicação e os tópicos.

Comparativo de Protocolos (MQTT vs. HTTP vs. CoAP)

Estabelecemos as seguintes diferenças entre os protocolos:

- **Modelo de comunicação:** O MQTT utiliza um modelo assíncrono de Publish/Subscribe (Publicação/Assinatura). O HTTP funciona em um modelo síncrono de Cliente-Servidor baseado em requisição-resposta. Já o CoAP opera em um modelo Cliente-Servidor assíncrono, também baseado em requisição-resposta.
- **Eficiência de banda e tamanho de payload:** O MQTT possui alta eficiência de banda, operando com mensagens compactas e cabeçalhos mínimos. O HTTP apresenta uma eficiência moderada a baixa devido à presença de cabeçalhos pesados, dificultando o uso em redes restritas. O CoAP possui alta eficiência e foi projetado para mensagens pequenas em redes restritas.
- **Consumo de energia:** O MQTT apresenta baixo consumo de energia, sendo otimizado para dispositivos IoT alimentados por bateria. O HTTP possui um alto consumo de energia, uma vez que suas conexões frequentes exigem mais recursos. O CoAP tem um consumo de energia muito baixo, sendo ideal para dispositivos com restrições extremas de energia.



Visão Geral de Padrões Industriais

O MQTT tornou-se um padrão amplamente adotado em aplicações de IoT, sistemas distribuídos e monitoramento industrial. Na arquitetura moderna da Indústria 4.0, o MQTT atua como uma ponte crucial entre o chão de fábrica e os sistemas de Tecnologia da Informação (TI). Ele facilita a integração de dados coletados por sensores e PLCs com sistemas SCADA (Supervisory Control and Data Acquisition), permitindo que essas informações sejam enviadas de forma leve, resiliente e escalável para arquiteturas em nuvem.

Para garantir uma comunicação robusta na indústria, o MQTT é frequentemente utilizado ao lado do padrão **OPC (Open Platform Communications)**, mais especificamente o OPC UA (Unified Architecture). Enquanto o MQTT resolve o problema de "como" transportar os dados de maneira eficiente e leve até a nuvem, o padrão OPC atua na interoperabilidade e modelagem da informação, garantindo que máquinas de diferentes fabricantes consigam se comunicar de forma padronizada. Em muitas arquiteturas industriais, o OPC UA é usado para organizar e estruturar os dados localmente, e o MQTT é empregado para publicar esses dados em sistemas remotos e plataformas em nuvem.

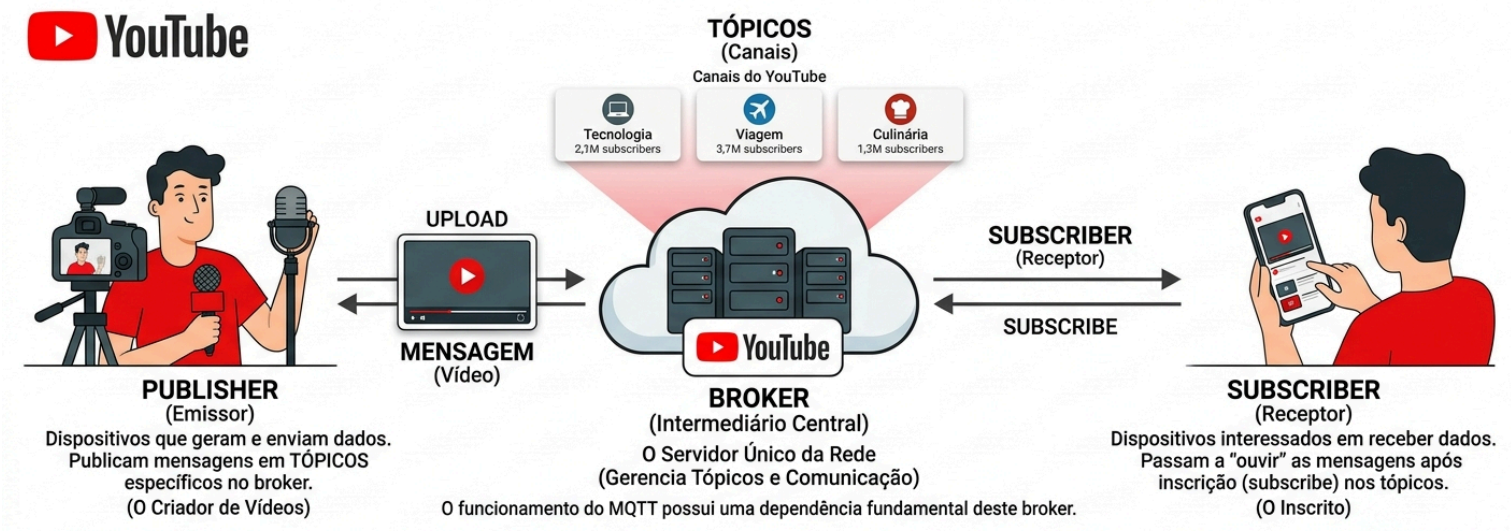
---

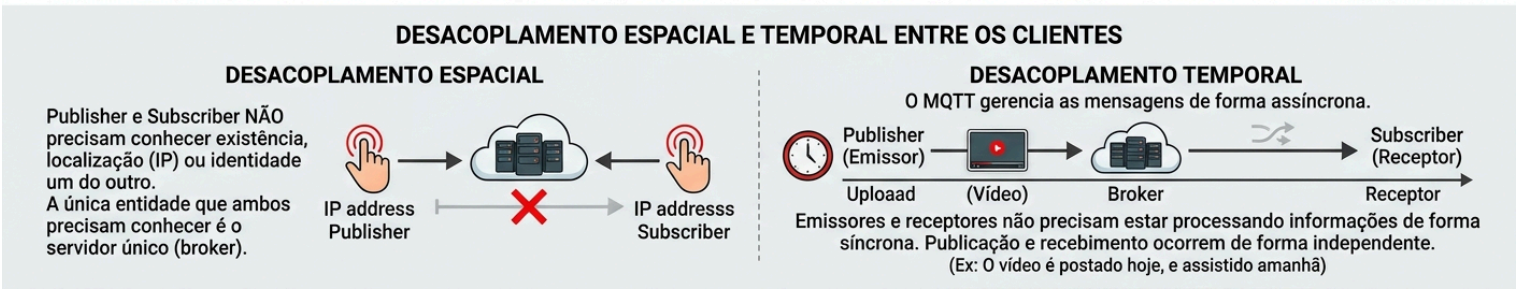
2. Arquitetura Publish/Subscribe (MQTT)

Fundamentos do Modelo Pub/Sub

A arquitetura do protocolo MQTT utiliza o paradigma Publish/Subscribe (Publicar/Assinar), caracterizando-se por ser um modelo orientado a eventos. Diferente do modelo tradicional de requisição-resposta, essa abordagem transforma a maneira como os nós da rede interagem, baseando-se em três componentes principais:

- **Componentes e papéis centrais:**
  - **Broker (Intermediário central):** Atua como o servidor único da rede, sendo a entidade central responsável pelo gerenciamento dos tópicos e de toda a comunicação entre os dispositivos. O funcionamento do MQTT possui uma dependência fundamental desse broker.
  - **Publisher (Emissor):** Consiste nos dispositivos conectados como clientes que geram e enviam os dados. Esses dispositivos publicam as suas mensagens direcionando-as a tópicos específicos no broker.
  - **Subscriber (Receptor):** Consiste nos dispositivos clientes interessados em receber determinados dados. Eles passam a "ouvir" as mensagens de outros dispositivos após realizarem o processo de inscrição (subscribe) nos tópicos gerenciados pelo broker.
- **Desacoplamento espacial e temporal entre os clientes:** O modelo Publish/Subscribe promove um alto nível de isolamento entre quem envia e quem recebe os dados, o que é conhecido como desacoplamento. Como todos os dispositivos se conectam unicamente ao broker e não diretamente entre si, temos:
  - **Desacoplamento Espacial:** O Publisher e o Subscriber não precisam conhecer a existência, a localização na rede (IP) ou a identidade um do outro. A única entidade que ambos precisam conhecer e acessar é o servidor único (o broker).
  - **Desacoplamento Temporal:** Embora o MQTT seja excelente para comunicação em tempo real, os emissores e receptores não precisam estar processando informações de forma síncrona. O gerenciamento das mensagens pelo broker garante que a publicação e o recebimento ocorram de forma independente e assíncrona.





Estrutura Hierárquica de Tópicos e Wildcards

Organização de canais via strings hierárquicas

No protocolo MQTT, as mensagens não são enviadas diretamente de um nó para outro, mas sim direcionadas a "tópicos", que funcionam como endereços ou canais de comunicação gerenciados pelo *broker*.

- **Estrutura de Strings:** Os tópicos são organizados em formato de árvore lógica, utilizando strings delimitadas pelo caractere de barra (`/`) para criar uma hierarquia simples e intuitiva.
- **Exemplo Prático:** Uma estrutura de tópicos bem definida em uma automação residencial pode seguir a taxonomia:
  - `casa/sala/temperatura`
  - `casa/sala/umidade`
  - `casa/cozinha/luz`
- **Sensibilidade a Maiúsculas:** É importante destacar que os tópicos são *case-sensitive* (diferenciam maiúsculas de minúsculas). Portanto, `casa/sala/temperatura` e `Casa/Sala/Temperatura` representam canais totalmente **distintos** no *broker*.

Uso de Wildcards para Subscrição Flexível

Para evitar que um cliente precise se inscrever individualmente em dezenas ou centenas de tópicos específicos, o MQTT oferece o recurso de **wildcards** (caracteres coringa). Eles só podem ser utilizados no processo de **inscrição (subscription)**, sendo proibidos durante a publicação de mensagens.

1. **Wildcard de Nível Único (`+`):**
   - Substitui exatamente um único nível da hierarquia de tópicos.
   - **Exemplo:** Ao se inscrever no tópico `casa/+/temperatura`, o cliente receberá mensagens de:
     - `casa/sala/temperatura`
     - `casa/cozinha/temperatura`
     - `casa/quarto/temperatura`
   - *Nota:* Ele não responderá a `casa/temperatura` ou `casa/primeiro-andar/sala/temperatura`, pois substitui estritamente apenas *um nível* entre as barras.

2. **Wildcard de Múltiplos Níveis (`#`):**
   - Substitui qualquer número de níveis subsequentes na hierarquia de tópicos.
   - **Regra de Uso:** Deve sempre ser posicionado no final da string do tópico e ser precedido por uma barra `/` (ou ser usado sozinho).
   - **Exemplo:** Ao se inscrever no tópico `casa/sala/#`, o cliente receberá qualquer publicação que comece com o caminho especificado:
     - `casa/sala/temperatura`
     - `casa/sala/iluminacao/lustre`
     - `casa/sala/ar-condicionado/status/erro`
   - Se um cliente se inscrever no tópico raiz `#`, ele passará a receber **todas** as mensagens que trafegam por aquele *broker* (útil para auditoria, *dashboards* ou gravação em banco de dados).

Níveis de Qualidade de Serviço (QoS)

No protocolo MQTT, a Qualidade de Serviço (QoS) tem um significado diferente das redes de roteadores tradicionais. Em vez de lidar com filas e priorização de tráfego, o QoS no MQTT define o **nível de garantia de entrega** de uma mensagem entre o cliente (publisher ou subscriber) e o broker. O MQTT possui três níveis exatos de QoS, equilibrando confiabilidade e consumo de recursos:

QoS 0: No máximo uma vez (At most once)

- **Como funciona:** É o modelo "dispara e esquece" (*fire and forget*). O remetente envia a mensagem apenas uma vez e não exige nenhuma confirmação de recebimento do broker.
- **Características:** É o nível mais rápido, que consome menos banda e bateria, mas não oferece nenhuma garantia. Se a conexão falhar no momento do envio, a mensagem é perdida.
- **Uso ideal:** Dados não críticos que são atualizados com alta frequência, onde perder uma amostra não importa (ex: um sensor de temperatura que envia leituras a cada 2 segundos).

QoS 1: Pelo menos uma vez (At least once)

- **Como funciona:** Garante que a mensagem chegará ao destino, mas pode resultar em entregas duplicadas. O remetente guarda a mensagem e continua reenviando-a até receber um pacote de confirmação (`PUBACK`) do receptor.
- **Características:** Consome um pouco mais de banda e recursos. Se houver lentidão na rede e o `PUBACK` demorar a chegar, o remetente reenvia o pacote, fazendo com que o broker (ou cliente) receba a mesma mensagem mais de uma vez.
- **Uso ideal:** Sistemas onde perder a informação é inaceitável, mas a aplicação final consegue lidar com dados duplicados sem gerar erros (operações idempotentes, como "ligar luz", onde receber o comando "ligar" duas vezes não muda o estado final).

QoS 2: Exatamente uma vez (Exactly once)

- **Como funciona:** É o nível mais alto, mais seguro e também o mais lento. Utiliza um processo de confirmação em quatro etapas (*4-way handshake*: `PUBLISH`, `PUBREC`, `PUBREL`, `PUBCOMP`) para garantir que a mensagem chegue uma única vez, sem perdas e sem duplicatas.
- **Características:** Consome significativamente mais banda e aumenta a latência da comunicação.
- **Uso ideal:** Comandos críticos ou sistemas onde uma duplicidade causaria problemas graves e cumulativos (ex: cobrança financeira, sistema de contagem, acionamento crítico de uma máquina).

Detalhe importante de arquitetura: O QoS no MQTT é negociado em duas etapas separadas:

1. Do dispositivo que publica (*Publisher*) para o *Broker*.
2. Do *Broker* para o dispositivo que assina o tópico (*Subscriber*).

O nível de QoS final, de ponta a ponta, será sempre limitado ao **menor** nível de QoS configurado entre essas duas conexões. Se um dispositivo publicar em QoS 2, mas o assinante estiver configurado para receber em QoS 0, a entrega final para esse assinante será em QoS 0.

---

3. Mecanismos de Resiliência e Segurança

Recurso Last Will and Testament (LWT)

O Last Will and Testament (Testamento e Última Vontade) é um recurso fundamental de resiliência no MQTT, concebido para mitigar problemas em conectividades intermitentes. O *broker* MQTT rastreia ativamente os clientes conectados à rede utilizando um sistema de mensagens de *keep-alive*. Ao se conectar ao servidor, um dispositivo pode registrar antecipadamente uma mensagem de "testamento" vinculada a um tópico específico. Caso o dispositivo sofra uma desconexão abrupta e não intencional (como perda de bateria, rompimento de

cabo ou falha de rede) e pare de enviar os pacotes *keep-alive*, o *broker* detecta a falha e publica automaticamente a mensagem LWT registrada. Isso garante que o restante do sistema, como painéis de monitoramento ou sistemas de alarme, seja imediatamente notificado sobre a indisponibilidade repentina do nó.

Segurança em Camadas

O protocolo MQTT foi projetado visando a máxima eficiência de processamento e consumo de banda, o que significa que ele não possui criptografia de dados habilitada por padrão. Para garantir implementações seguras em IoT, adota-se um modelo de segurança baseado em camadas:

- **Autenticação por Usuário e Senha:** O servidor MQTT atua como o hub central responsável pela autenticação e autorização dos clientes MQTT. A primeira etapa obrigatória para que qualquer dispositivo possa atuar como emissor (*publisher*) ou receptor (*subscriber*) é estabelecer a conexão fornecendo credenciais válidas ao servidor.
- **Criptografia na Camada de Transporte (TLS/SSL):** Como o protocolo MQTT é executado sobre a pilha TCP/IP, a proteção contra interceptação de dados (ataques *man-in-the-middle*) é feita criptografando o canal de comunicação na camada de transporte. A aplicação de TLS/SSL gera o que convencionalmente chamamos de MQTTS (MQTT over TLS), garantindo a confidencialidade e a integridade da carga útil (*payload*) das mensagens durante a transmissão.
- **Controle de Acesso via ACLs (Access Control Lists):** Após a etapa de autenticação, o servidor MQTT precisa gerenciar a autorização dos dispositivos conectados. As Listas de Controle de Acesso (ACLs) definem permissões granulares, determinando exatamente quais tópicos cada usuário pode acessar, bem como o tipo de operação permitida (leitura/assinatura ou escrita/publicação). Isso garante, por exemplo, que um simples sensor de umidade tenha permissão apenas para publicar em seu tópico específico, impossibilitando-o de enviar comandos críticos para o sistema de refrigeração.

---

4. Simulação de Protocolos MQTT/OPC

Passo 1: Configurar a Conexão com o Broker

1. Abra o **MQTT.fx**.
2. Clique no ícone de **engrenagem** (ao lado do botão *Connect*).
3. Clique no botão **+** (canto inferior esquerdo) para criar um novo perfil.
4. Preencha os campos com as seguintes informações:
   - **Profile Name:** Broker Publico HiveMQ
   - **Broker Address:** ://hivemq.com
   - **Broker Port:** 1883
5. Clique em **Apply** e depois em **OK**.
6. Na tela principal, certifique-se de que o perfil criado está selecionado e clique em **Connect**.
   - *O indicador redondo no canto superior direito deverá ficar **verde**, confirmando a conexão.*

Passo 2: Inscrever-se em um Tópico (Subscribe)

Antes de enviar uma mensagem, precisamos de alguém para escutá-la. Vamos simular um dispositivo receptor (como um painel de controle ou um aplicativo).

1. Clique na aba **Subscribe** (Inscrição).

2. No campo de texto, digite um tópico exclusivo para evitar misturar com mensagens de outros usuários do broker público. Exemplo: meuteste/mqtt/sensor1
3. Clique no botão **Subscribe** à direita.
4. O tópico aparecerá na lista logo abaixo, indicando que o MQTT.fx está "ouvindo" esse canal.

Passo 3: Publicar uma Mensagem (Publish)
Agora vamos simular um dispositivo transmissor (como um sensor de temperatura).
1. Clique na aba **Publish** (Publicação).
2. No campo de texto do tópico, insira **exatamente o mesmo tópico** que você escolheu no Passo 2: meuteste/mqtt/sensor1

No campo de texto grande abaixo (área da mensagem/payload), digite um dado em formato JSON ou texto simples. Exemplo:
json
```
{
  "temperatura": 24.5,
  "status": "normal"
}
```
3. Use o código com cuidado.
4. Clique no botão **Publish**.

Passo 4: Verificar o Recebimento
1. Volte para a aba **Subscribe**.
2. Olhe para o lado direito da tela. Você verá a mensagem enviada, acompanhada da data/hora e do tópico correspondente.

Desafio Extra: Entendendo o desacoplamento do MQTT
Para fixar o aprendizado, faça o seguinte teste:
1. Vá na aba **Subscribe** e clique em **Unsubscribe** para cancelar a inscrição.
2. Vá na aba **Publish** e mude o valor da temperatura para 30.0. Clique em **Publish**.
3. Volte na aba **Subscribe** e observe que nada foi recebido (já que você não estava inscrito no momento do envio). O MQTT funciona em tempo real e não armazena mensagens por padrão (a menos que configurado com *Retained Messages* ou *QoS 1/2 com sessões persistentes*).

Dinâmica em grupo
Para que os colegas de sala consigam se inscrever nos tópicos uns dos outros, o segredo está na padronização dos nomes dos tópicos. Como todos estão conectados ao **mesmo broker público** (://hivemq.com), qualquer pessoa na sala pode ouvir ou falar com qualquer colega, desde que saiba o nome exato do canal. Aqui está como configurar essa dinâmica em grupo:

1. Criem um Padrão de Tópico para a Sala
Para evitar que mensagens de desconhecidos na internet entrem no exercício, combinem uma estrutura única. Por exemplo: sala3a/nome_do_aluno/sensor

2. Exemplo Prático de Interação (Aluno A e Aluno B)

Imaginem que o **Aluno A** se chama *Carlos* e o **Aluno B** se chama *Mariana*.

- **O que o Carlos (Aluno A) faz:**
  1. Vai na aba **Publish**.
  2. Digita o tópico dele: sala3a/carlos/temperatura
  3. Digita a mensagem: {"temp": 22} e clica em **Publish**.
- **O que a Mariana (Aluno B) faz para ouvir o Carlos:**
  1. Vai na aba **Subscribe**.
  2. Digita exatamente o tópico do Carlos: sala3a/carlos/temperatura
  3. Clica em **Subscribe**. Pronto, ela verá as mensagens que o Carlos enviar.

3. O "Modo Professor" (Ouvir a sala toda de uma vez)

Se você ou algum colega quiser receber as mensagens de **todos os alunos da sala ao mesmo tempo**, o MQTT permite o uso de um caractere curinga chamado **Wildcard (#)**.

1. Vá na aba **Subscribe**.
2. Digite o tópico usando a hashtag no final: sala3a/#
3. Clique em **Subscribe**.

O símbolo # diz ao broker: *"Me envie qualquer mensagem que comece com sala3a/, não importa o que venha depois"*. Assim, você verá as publicações do Carlos, da Mariana e de qualquer outro colega instantaneamente.